

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №6

Реализация защиты от CSRF/XSS. Работа с secure cookies

Цель занятия

Освоить базовые практические подходы к защите web-приложения на Go от CSRF- и XSS-угроз, а также научиться безопасно использовать cookies для аутентификации и хранения пользовательского состояния.

Задачи занятия

1. Изучить природу атак CSRF и XSS и понять, в каких сценариях они возникают.
2. Разобрать различие между хранением токена в cookie и передачей токена в заголовках или теле запроса.
3. Освоить использование защитных свойств cookies: `HttpOnly`, `Secure`, `SameSite`.
4. Научиться генерировать и проверять CSRF-токен в backend-приложении.
5. Реализовать форму и backend-обработчик с проверкой CSRF-токена.
6. Понять, почему вывод пользовательских данных без экранирования создаёт риск XSS.
7. Научиться безопасно отображать пользовательские данные в HTML-шаблоне.
8. Реализовать учебный пример сервиса с авторизационной cookie, CSRF-защитой и безопасным выводом HTML.

Теоретическая часть

1. Почему backend-разработчику нужно думать о web-безопасности

Даже если backend-сервис кажется простым, любая работа с браузером, HTML-страницами, формами, cookies и пользовательским вводом создаёт поверхность атаки. На практике очень многие уязвимости возникают не потому, что система сложная, а потому, что в ней не были учтены базовые принципы защиты: проверка происхождения запроса, безопасная работа с cookie, экранирование пользовательского содержимого, ограничение доверия к данным, пришедшим из браузера.

В этой работе будут рассмотрены две классические угрозы: CSRF и XSS.

2. Что такое CSRF

CSRF — это ситуация, когда браузер пользователя отправляет запрос от его имени на целевой сайт без осознанного намерения самого пользователя. Это возможно потому, что браузер автоматически прикладывает cookies к запросу, если сайт и политика браузера это позволяют.

Пример учебного сценария:

- пользователь авторизован на сайте;
- в браузере уже есть cookie сессии;
- злоумышленник заставляет браузер пользователя открыть специальную страницу или отправить форму;
- браузер автоматически прикладывает cookie;
- сервер видит «авторизованный» запрос и выполняет действие.

Главная проблема CSRF в том, что сам запрос может выглядеть корректно с точки зрения сервера, если сервер не проверяет дополнительный признак подлинности.

3. Что такое XSS

XSS – это внедрение вредоносного клиентского кода, чаще всего JavaScript, в страницу, которую затем увидит пользователь в браузере.

Классический сценарий:

- пользователь отправляет комментарий или имя;
- сервер сохраняет текст;
- затем этот текст выводится в HTML без экранирования;
- браузер интерпретирует содержимое не как текст, а как HTML или script.

В результате злоумышленник может подменять интерфейс, воровать данные, отправлять действия от лица пользователя, читать доступное содержимое страницы или мешать нормальной работе приложения.

4. Чем CSRF отличается от XSS

Эти угрозы часто путают, но по сути они разные.

CSRF использует доверие сервера к браузеру пользователя. Сервер ошибочно считает запрос легитимным, потому что в нём есть нужная cookie.

XSS использует доверие браузера к данным, пришедшим от сервера. Браузер ошибочно считает вредоносный фрагмент частью нормальной страницы и выполняет его.

Иными словами:

- CSRF атакует механику авторизованного запроса;
- XSS атакует отображение данных в браузере.

5. Как защищаются от CSRF

Для защиты от CSRF обычно используют несколько подходов вместе:

- CSRF-токен, который должен присутствовать в форме и проверяться на сервере;
- настройку SameSite у cookies;
- ограничение опасных действий только методами POST, PUT, PATCH, DELETE;
- проверку Origin или Referer в более строгих сценариях.

В данной работе будет реализован базовый и понятный вариант: CSRF-токен в форме и проверка этого токена на сервере.

6. Как защищаются от XSS

Основная защита от XSS — не выводить пользовательские данные в HTML как «готовую разметку», если они пришли извне и им нельзя полностью доверять.

В учебном приложении это означает:

- использовать шаблоны, которые экранируют HTML автоматически;
- не вставлять пользовательский ввод в страницу вручную через конкатенацию HTML-строк;
- не использовать небезопасные типы или «сырой HTML» без крайней необходимости.

7. Что такое secure cookies

Cookie – это маленький фрагмент данных, который сервер отдаёт браузеру, а браузер затем может автоматически отправлять обратно.

Для безопасности важны следующие атрибуты:

HttpOnly – запрещает доступ к cookie из JavaScript в браузере. Это снижает риск кражи cookie через XSS.

Secure – говорит браузеру отправлять cookie только по HTTPS.

SameSite – ограничивает передачу cookie в межсайтовых сценариях. Это помогает уменьшить риск CSRF.

Для учебной работы важно не просто создать cookie, а создать её безопасно.

8. Что мы сделаем в этой работе

В данной практике будет реализовано небольшое web-приложение на Go со следующими маршрутами:

- GET /login – имитация входа и установка авторизационной cookie;
- GET /profile – форма изменения отображаемого имени пользователя;
- POST /profile – обновление имени после проверки CSRF-токена;
- GET /hello – HTML-страница, отображающая имя пользователя безопасным способом.

В приложении будут реализованы:

- secure cookies;
- простая CSRF-защита;
- безопасный вывод HTML через шаблоны;
- демонстрация того, как может выглядеть опасный XSS-подход и чем он отличается от безопасного.

Практическая часть

Общая идея проекта

В этой работе создаётся мини-приложение с имитацией авторизованного пользователя.

После «входа» пользователю устанавливается cookie сессии.

При открытии профиля сервер выдаёт HTML-форму и подставляет в неё CSRF-токен.

Когда форма отправляется, сервер:

- проверяет наличие авторизационной cookie;
- проверяет CSRF-токен;
- обновляет имя пользователя.

Затем имя отображается на отдельной странице безопасно, через HTML-шаблон.

Рекомендуемая структура проекта

```
pz6-web-security/  
  cmd/  
    server/  
      main.go  
  internal/  
    auth/  
      cookie.go  
      csrf.go  
    httpapi/  
      handler.go  
    store/  
      store.go  
  templates/  
    profile.html
```

```
hello.html  
go.mod
```

Шаг 1. Создание проекта

Откройте PowerShell и создайте папку проекта:

```
mkdir pz6-web-security  
cd pz6-web-security  
go mod init example.com/pz6-web-security  
mkdir cmd  
mkdir cmd\server  
mkdir internal  
mkdir internal\auth  
mkdir internal\httpapi  
mkdir internal\store  
mkdir templates
```

Для macOS/Linux:

```
mkdir -p pz6-web-security/cmd/server  
mkdir -p pz6-web-security/internal/auth  
mkdir -p pz6-web-security/internal/httpapi  
mkdir -p pz6-web-security/internal/store  
mkdir -p pz6-web-security/templates  
cd pz6-web-security  
go mod init example.com/pz6-web-security
```

Шаг 2. Создание простого in-memory хранилища

Создайте файл `internal/store/store.go`:

```
package store  
  
import "sync"
```



```
type UserProfile struct {
    SessionID string
    Name      string
    CSRFToken string
}

type Store struct {
    mu      sync.RWMutex
    users map[string]*UserProfile
}

func New() *Store {
    return &Store{
        users: make(map[string]*UserProfile),
    }
}

func (s *Store) Save(profile *UserProfile) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.users[profile.SessionID] = profile
}

func (s *Store) Get(sessionID string) (*UserProfile,
bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    profile, ok := s.users[sessionID]
    return profile, ok
}
```

```

func (s *Store) UpdateName(sessionID, name string) bool
{
    s.mu.Lock()
    defer s.mu.Unlock()

    profile, ok := s.users[sessionID]
    if !ok {
        return false
    }

    profile.Name = name
    return true
}

```

Шаг 3. Генерация идентификаторов и CSRF-токенов

Создайте файл `internal/auth/csrf.go`:

```

package auth

import (
    "crypto/rand"
    "encoding/hex"
)

func RandomToken(size int) (string, error) {
    buf := make([]byte, size)
    if _, err := rand.Read(buf); err != nil {
        return "", err
    }
    return hex.EncodeToString(buf), nil
}

```

Пояснение

Для сессии и CSRF-токена нужны трудноугадываемые значения. В учебной практике подойдёт генерация случайного токена через криптографически стойкий источник случайных байтов.

Шаг 4. Работа с cookie

Создайте файл `internal/auth/cookie.go`:

```
package auth

import "net/http"

const SessionCookieName = "session_id"

func SetSessionCookie(w http.ResponseWriter, value string) {
    http.SetCookie(w, &http.Cookie{
        Name:     SessionCookieName,
        Value:    value,
        Path:     "/",
        HttpOnly: true,
        Secure:   false,
        SameSite: http.SameSiteLaxMode,
        MaxAge:   3600,
    })
}

func ReadSessionCookie(r *http.Request) (string, error) {
    cookie, err := r.Cookie(SessionCookieName)
    if err != nil {
        return "", err
    }
}
```

```
}  
    return cookie.Value, nil  
}
```

Пояснение

Здесь:

- `HttpOnly: true` запрещает чтение cookie из JavaScript;
- `SameSiteLaxMode` даёт базовую защиту от части межсайтовых сценариев;
- `Secure` пока установлен в `false`, потому что в локальной учебной среде приложение будет запускаться по обычному HTTP.

Если объединять это ПЗ с предыдущим ПЗ по HTTPS, то в защищённой среде нужно обязательно ставить `Secure: true`.

Шаг 5. HTML-шаблон формы профиля

Создайте файл `templates/profile.html`:

```
<!DOCTYPE html>  
<html lang="ru">  
  <head>  
    <meta charset="UTF-8">  
    <title>Профиль</title>  
  </head>  
  <body>  
    <h1>Редактирование профиля</h1>  
  
    <p>Текущее имя: <strong>{{.Name}}</strong></p>  
  
    <form method="POST" action="/profile">  
      <input type="hidden" name="csrf_token"  
value="{{.CSRFToken}}">
```

```
        <label for="name">Новое имя:</label>
        <input type="text" id="name" name="name"
value="{{ .Name }}">
        <button type="submit">Сохранить</button>
    </form>

    <p><a href="/hello">Открыть страницу
приветствия</a></p>
</body>
</html>
```

Шаг 6. HTML-шаблон приветствия

Создайте файл `templates/hello.html`:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Приветствие</title>
</head>
<body>
    <h1>Здравствуйтесь, {{ .Name }}!</h1>
    <p>Это безопасный вывод имени пользователя через
шаблон.</p>
    <p><a href="/profile">Вернуться к профилю</a></p>
</body>
</html>
```

Пояснение

Здесь имя пользователя выводится через шаблон, а не вставляется в HTML вручную. Это важный шаг для защиты от XSS.

Шаг 7. Реализация обработчиков

Создайте файл `internal/httpapi/handler.go`:

```
package httpapi

import (
    "html/template"
    "net/http"
    "strings"

    "example.com/pz6-web-security/internal/auth"
    "example.com/pz6-web-security/internal/store"
)

type Handler struct {
    store          *store.Store
    profileTmpl    *template.Template
    helloTmpl      *template.Template
}

func NewHandler(s *store.Store) (*Handler, error) {
    profileTmpl, err :=
template.ParseFiles("templates/profile.html")
    if err != nil {
        return nil, err
    }

    helloTmpl, err :=
template.ParseFiles("templates/hello.html")
    if err != nil {
        return nil, err
    }
}
```

```

    return &Handler{
        store:      s,
        profileTpl: profileTpl,
        helloTpl:    helloTpl,
    }, nil
}

func (h *Handler) Login(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    sessionID, err := auth.RandomToken(16)
    if err != nil {
        http.Error(w, "failed to create session",
http.StatusInternalServerError)
        return
    }

    csrfToken, err := auth.RandomToken(16)
    if err != nil {
        http.Error(w, "failed to create csrf token",
http.StatusInternalServerError)
        return
    }

    h.store.Save(&store.UserProfile{
        SessionID: sessionID,

```

```

        Name:      "Студент",
        CSRFToken: csrfToken,
    })

    auth.SetSessionCookie(w, sessionID)

    http.Redirect(w, r, "/profile", http.StatusFound)
}

func (h *Handler) Profile(w http.ResponseWriter, r
*http.Request) {
    sessionID, err := auth.ReadSessionCookie(r)
    if err != nil {
        http.Error(w, "unauthorized",
http.StatusUnauthorized)
        return
    }

    profile, ok := h.store.Get(sessionID)
    if !ok {
        http.Error(w, "session not found",
http.StatusUnauthorized)
        return
    }

    switch r.Method {
    case http.MethodGet:
        data := struct {
            Name      string
            CSRFToken string
        }{
            Name:      profile.Name,

```



```

        CSRFToken: profile.CSRFToken,
    }

    if err := h.profileTmpl.Execute(w, data); err != nil
{
        http.Error(w, "template error",
http.StatusInternalServerError)
        return
    }

    case http.MethodPost:
        if err := r.ParseForm(); err != nil {
            http.Error(w, "bad form",
http.StatusBadRequest)
            return
        }

        tokenFromForm := r.FormValue("csrf_token")
        if tokenFromForm == "" || tokenFromForm !=
profile.CSRFToken {
            http.Error(w, "invalid csrf token",
http.StatusForbidden)
            return
        }

        name := strings.TrimSpace(r.FormValue("name"))
        if name == "" {
            http.Error(w, "name is required",
http.StatusBadRequest)
            return
        }

```

```

        h.store.UpdateName(sessionID, name)
        http.Redirect(w, r, "/hello", http.StatusFound)

    default:
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
    }
}

func (h *Handler) Hello(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
        return
    }

    sessionID, err := auth.ReadSessionCookie(r)
    if err != nil {
        http.Error(w, "unauthorized",
http.StatusUnauthorized)
        return
    }

    profile, ok := h.store.Get(sessionID)
    if !ok {
        http.Error(w, "session not found",
http.StatusUnauthorized)
        return
    }

    data := struct {

```

```

        Name string
    }{
        Name: profile.Name,
    }

    if err := h.helloTmpl.Execute(w, data); err != nil {
        http.Error(w, "template error",
http.StatusInternalServerError)
        return
    }
}

```

Шаг 8. Точка входа приложения

Создайте файл `cmd/server/main.go`:

```

package main

import (
    "log"
    "net/http"

    "example.com/pz6-web-security/internal/httpapi"
    "example.com/pz6-web-security/internal/store"
)

func main() {
    st := store.New()

    handler, err := httpapi.NewHandler(st)
    if err != nil {
        log.Fatal(err)
    }
}

```

```
    mux := http.NewServeMux()
    mux.HandleFunc("/login", handler.Login)
    mux.HandleFunc("/profile", handler.Profile)
    mux.HandleFunc("/hello", handler.Hello)

    log.Println("server started on http://localhost:8080")
    log.Println("open http://localhost:8080/login")

    if err := http.ListenAndServe(":8080", mux); err !=
nil {
        log.Fatal(err)
    }
}
```

Шаг 9. Запуск приложения

В корне проекта выполните:

```
go run ./cmd/server
```

Ожидаемый результат:

```
server started on http://localhost:8080
open http://localhost:8080/login
```

Шаг 10. Проверка сценария входа и установки cookie

Откройте в браузере:

```
http://localhost:8080/login
```

После этого приложение должно:

- создать сессию;
- установить cookie `session_id`;
- перенаправить пользователя на `/profile`.

Если всё выполнено правильно, откроется страница редактирования профиля.

Шаг 11. Проверка CSRF-защиты

На странице `/profile`:

- в скрытом поле формы уже будет встроен CSRF-токен;
- при нажатии кнопки «Сохранить» браузер отправит POST `/profile`;
- сервер проверит токен;
- при корректном токене обновит имя и перенаправит на `/hello`.

Попробуйте ввести обычное имя, например:

```
Иван
```

После отправки должно открыться приветствие:

```
Здравствуйте, Иван!
```

Шаг 12. Проверка ошибки CSRF

Теперь нужно увидеть, что защита действительно работает.

Самый простой учебный способ — открыть инструменты разработчика браузера или перехватить запрос в Postman/Insomnia, а затем отправить POST `/profile` без `csrf_token` или с неверным токеном.

Если токен неправильный, сервер должен вернуть:

```
403 Forbidden
invalid csrf token
```

Это означает, что наличие одной только cookie больше не является достаточным условием для выполнения действия.

Шаг 13. Демонстрация XSS-риска

Попробуйте ввести в поле имени такой текст:

```
<script>alert('xss')</script>
```

После сохранения приложение откроет страницу /hello.

При безопасной реализации вы должны увидеть сам текст, а не выполнение скрипта. То есть страница не должна запускать `alert`, а должна отобразить введённую строку как обычное содержимое.

Именно это и показывает, что шаблон выводит данные безопасно.

Шаг 14. Как выглядел бы опасный XSS-вариант

Ниже приведён пример опасного подхода. Его **не нужно** использовать в рабочем коде. Он нужен только для понимания проблемы.

```
func unsafeHello(w http.ResponseWriter, name string) {
    html := "<html><body><h1>Здравствуйте, " + name +
"!</h1></body></html>"
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    _, _ = w.Write([]byte(html))
}
```

Если в `name` попадёт HTML или `script`, браузер может интерпретировать это как часть страницы.

Поэтому пользовательские данные нельзя вставлять в HTML вручную через конкатенацию строк.

Что получилось в результате

В ходе практической работы реализовали учебное web-приложение, в котором:

- используется cookie для хранения идентификатора сессии;
- cookie настроена безопаснее, чем обычная «голая» cookie;
- форма защищена CSRF-токеном;
- сервер проверяет токен перед изменением данных;
- имя пользователя отображается через шаблон безопасным способом;
- XSS-угроза разобрана на контрасте между опасным и безопасным подходом.

Что важно понять по итогам работы

CSRF и XSS — это не «экзотические» угрозы, а очень практические классы уязвимостей.

CSRF возникает там, где сервер излишне доверяет автоматической отправке cookies браузером.

XSS возникает там, где приложение излишне доверяет данным, которые потом показывает в HTML.

Безопасное backend-приложение должно:

- осторожно использовать cookies;
- не считать наличие cookie достаточным доказательством намерения пользователя;
- экранировать пользовательский ввод при выводе в HTML;
- минимизировать доверие к данным со стороны клиента.

Дополнительные задания

Вариант 1. Сделать logout

Добавьте маршрут:

```
GET /logout
```

который очищает session cookie и завершает «сессию» пользователя.

Вариант 2. Включить Secure для cookie

Если вы объединяете эту практику с HTTPS из предыдущего занятия, установите `Secure: true` у cookie и запустите приложение по HTTPS.

Вариант 3. Ротация CSRF-токена

После успешного `POST /profile` генерируйте новый CSRF-токен и сохраняйте его в профиле.

Вариант 4. Добавить страницу комментариев

Сделайте форму ввода комментария и безопасный вывод комментария через шаблон. Это поможет ещё лучше закрепить тему XSS.

Требования к отчёту

В отчёте должны быть представлены:

- тема и цель практической работы;
- краткое объяснение, что такое CSRF;
- краткое объяснение, что такое XSS;
- объяснение роли `HttpOnly`, `Secure`, `SameSite`;
- структура проекта;
- фрагменты кода:
 - установка cookie;
 - генерация CSRF-токена;
 - проверка CSRF-токена;

- безопасный HTML-шаблон;
- опасный XSS-пример;
- результаты проверки:
 - переход на `/login`;
 - отображение формы профиля;
 - успешное изменение имени;
 - ошибка при неверном CSRF-токене;
 - безопасное отображение строки со `script`-тегом;
- вывод о реализованных механизмах защиты.

Контрольные вопросы

1. Что такое CSRF?
2. Почему наличие `cookie` не гарантирует, что запрос действительно инициировал пользователь?
3. Что такое XSS?
4. Чем CSRF отличается от XSS?
5. Для чего нужен CSRF-токен?
6. Что делает атрибут `HttpOnly` у `cookie`?
7. Для чего нужен атрибут `Secure`?
8. Какую роль играет `SameSite`?
9. Почему нельзя вставлять пользовательский ввод в HTML через конкатенацию строк?
10. Почему шаблоны безопаснее ручной сборки HTML?

Типичные ошибки

Если после `/login` страница `/profile` отвечает `unauthorized`, нужно проверить, установилась ли `cookie` и отправляет ли её браузер обратно.

Если CSRF-проверка всегда падает, нужно убедиться, что:

- токен передаётся в скрытом поле формы;
- токен сохранён в серверном хранилище;
- сравнение выполняется именно для текущей сессии.

Если `script`-тег на странице действительно выполняется, значит данные выводятся небезопасно и где-то используется ручная сборка HTML вместо шаблона.

Если вы поставили `Secure: true`, но запускает приложение по обычному HTTP, браузер может не отправлять такую `cookie`. Это нормальное поведение.

Если `SameSite` вообще не указан, межсайтовые сценарии будут контролироваться хуже.